

SECURE DATA WIPING SYSTEM

Complete Technical Documentation

Generated: March 23, 2026

TABLE OF CONTENTS

1. Project Overview
2. Build & Run Instructions
3. Backend Architecture
4. Secure Wiping Algorithms
5. Frontend Architecture
6. Module Documentation
7. UI/UX Solutions
8. Threading & Performance
9. Security & Compliance
10. Future Enhancements

1. PROJECT OVERVIEW

Secure Data Wiping Desktop Application is a professional Windows desktop application designed for secure, auditable data destruction using military-grade algorithms. Built with PyQt6 framework and Python 3.10+, it provides compliance with GDPR, HIPAA, PCI-DSS, and DoD standards.

Key Features:

- 5 Government-Approved Wiping Algorithms (Simple, DoD, NIST, Gutmann, Crypto Erase)
- Complete Audit Trail & Compliance Logging
- Batch File Processing with Progress Tracking
- Free Space Wiping & External Drive Support
- Scheduled Wipe Operations
- Email-Based Compliance Reports
- Modern PyQt6 GUI with 11 Feature Pages
- Data Persistence & History Management
- Tamper-Proof Logging with Hash Chain
- Network Wiping Capabilities

2. BUILD & RUN INSTRUCTIONS

2.1 Prerequisites

Python 3.10 or higher, pip package manager, Windows operating system

2.2 Installation

- pip install -r requirements.txt
- Installs: PyQt6, PyInstaller, cryptography, reportlab, etc.

2.3 Development Mode

python secure_wipe_desktop.py

2.4 Build Executable

python build_exe.py (preferred) OR pyinstaller SecureWipe.spec

2.5 Output

./dist/SecureWipe.exe (Standalone executable)

3. BACKEND ARCHITECTURE

3.1 Core Components

wiper_core.py - Core wiping engine implementing 5 algorithms

- WipeAlgorithm enum: SIMPLE, DOD, NIST, GUTMANN, CRYPTO
- Supports individual files, directories, and free space
- Progress callbacks for UI updates
- Chunk-based processing (8192 bytes/chunk)

history_manager.py - Maintains wipe history and persistence

- JSONified wipe records with timestamps
- Performance metrics (speed, duration)
- File/directory tracking

audit_chain.py - Tamper-proof audit logging

- Hash-chain implementation (blockchain-style)
- Cryptographic signatures
- Immutable audit trail

verification_module.py - Post-wipe verification

- Confirms successful deletion
- Generates verification reports
- Hash comparison

4. SECURE WIPING ALGORITHMS

4.1 Simple Wipe (Single Pass Random)

Speed: ██████ (Fastest)

Security: ██ (Basic)

Passes: 1

Use Case: Quick deletion of non-sensitive data

Process:

1. Generate random bytes using `secrets.token_bytes()`
2. Overwrite file with random data in 8KB chunks
3. Sync to disk using `os.fsync()`
4. Delete file reference

Time Complexity: $O(n)$ where n = file size

Security Level: No recovery possible with basic tools

4.2 DoD 5220.22-M (Military Standard)

Speed: ████ (Moderate)

Security: ██████████ (Military Grade)

Passes: 3

Use Case: Government/Defense data, classified information

Process:

- Pass 1: Random bytes (`secrets.token_bytes()`)
- Pass 2: Zero bytes (`0x00`) - detects magnetic remanence
- Pass 3: One bytes (`0xFF`) - inverts magnetics
- Each pass: 8KB chunks with `os.fsync()` sync

Time Complexity: $O(3n) = O(n)$

Security Level: Exceeds NSA/DoD standards, reverses magnetic imprints

4.3 NIST SP 800-88 (SSD Optimized)

Speed: ██████ (Fast)

Security: ██████████ (Enterprise Grade)

Passes: 1

Use Case: Modern SSDs, enterprise environments

Process:

1. Single pass with cryptographically secure random data
2. Optimized for NAND flash memory (no multi-pass wear)
3. Proper chunk buffering
4. File system sync with integrity check

Why 1 Pass for SSD: SSD controllers use wear-leveling; cannot reliably overwrite same physical location multiple times

Time Complexity: $O(n)$

Security Level: Standards-compliant for SSDs, impossible recovery

4.4 Gutmann (Maximum Security)

Speed: ■ (Slowest)

Security: ■■■■■■■■■■ (Maximum)

Passes: 7

Use Case: Maximum security requirements, paranoid users

Process:

- Pass 1-4: Specific bit patterns (0x55, 0xAA, 0x92, 0x49)
- Pass 5-7: Random data with different seeds
- Each pass: Full file overwrite + fsync
- Total time = 7x of simple wipe

Historical Context: Originally designed to defeat MFM encoding artifacts. Modern HDDs have different encoding, but still provides maximum assurance against theoretical forensic recovery.

Time Complexity: $O(7n) = O(n)$

Security Level: Highest theoretical security for magnetic media

4.5 Cryptographic Erase (AES-256)

Speed: ■■■■■■ (Fastest)

Security: ■■■■■■■■■■ (Maximum)

Passes: 1 (XOR encryption)

Use Case: SSDs, cloud storage, encrypted partitions

Process:

1. Generate AES-256 encryption key
2. XOR file data with encrypted key stream
3. Original data mathematically unrecoverable
4. Key disposal = data disposal

Mathematical Security: If AES-256 is unbroken, data recovery is mathematically IMPOSSIBLE (not just practically difficult).

Comparison:

- Overwrite-based: Data is destroyed, but recovery possible with forensic tools if overwrite incomplete
- Crypto Erase: Original data NEVER exposed to unencryption; mathematically proven impossible to recover

Time Complexity: $O(n)$ with AES hardware acceleration

Security Level: Mathematically impossible recovery

5. FRONTEND ARCHITECTURE

5.1 Main Window Structure

secure_wipe_desktop.py - Main PyQt6 GUI Application

Architecture Pattern: QMainWindow + QStackedWidget

- Central widget contains horizontal layout
- Left sidebar: Navigation menu (QListWidget)
- Right side: QStackedWidget with 11 pages
- Status bar: Shows current operation status

Window Configuration:

- Size: 1200x800 (resizable)
- Theme: Dark (#0d1117 background)
- Accent: Green (#00e676)
- Font: Segoe UI, 10pt

5.2 Navigation Sidebar (11 Pages)

- 1. Dashboard - Overview, statistics, quick actions
- 2. Secure Wipe - Main file wiping interface
- 3. Free Space Wiper - Wipe unallocated disk space
- 4. External Drives - USB/SD card wiping
- 5. Batch Processing - Bulk file/folder wipe
- 6. Scheduled Wipe - Automated wipe scheduling
- 7. Audit & History - Complete operation logs
- 8. Verification Reports - Post-wipe verification
- 9. Settings - Configuration & preferences
- 10. Legal & Compliance - Terms, compliance info
- 11. About - Version, credits, help

5.3 Key UI Components

File Selection

- QListWidget: Multi-select, word-wrap, 280-500px height
- Buttons: Add Files, Add Folder, Remove, Clear All
- File count displayed dynamically

Algorithm Selection

- QComboBox dropdown with 5 algorithms
- Tooltip descriptions for each algorithm
- Real-time time estimate calculation

Progress Tracking

- QProgressBar: Overall operation progress
- QLabel: Current file being processed
- Speed, estimated time remaining

Operation Controls

- Start, Pause, Cancel buttons
- Confirmation dialogs before destructive actions
- Warning messages for user safety

6. MODULE DOCUMENTATION

6.1 Core Modules

wiper_core.py (300+ lines)

Purpose: Five secure wiping algorithms

Classes: WipeAlgorithm (enum), FileWiper

Key Methods: wipe_file(), wipe_directory(), wipe_free_space()

secure_wipe_desktop.py (1500+ lines)

Purpose: Main PyQt6 GUI application

Classes: SecureWipeApp (QMainWindow)

Key Methods: batch_add_files(), start_wipe(), update_progress()

history_manager.py

Purpose: Persistent wipe history storage

Data Format: JSON with timestamps, file paths, algorithms

Key Methods: add_record(), get_history(), export_report()

6.2 Security & Compliance Modules

audit_chain.py - Tamper-proof audit logging

Hash-chain implementation preventing log tampering

verification_module.py - Post-wipe verification

Confirms files are unrecoverable via hash comparison

certificate_generator.py - Compliance certificates

ReportLab PDF generation for compliance reports

crypto_utils.py - Cryptographic utilities

AES-256 encryption, SHA256 hashing, key generation

6.3 Feature Modules

batch_processor.py - Batch file processing

Queue management, parallel processing coordination

scheduled_wipe.py - Scheduled operations

Windows Task Scheduler integration

free_space_wiper.py - Unallocated space wiping

Recovers and wipes deleted file remnants

external_drives.py - USB/SD card support

External drive detection and wiping

network_wipe.py - Network device wiping

Remote device discovery and secure wiping

6.4 Support & Integration Modules

email_system.py - Compliance report generation & sending
notification_manager.py - User notifications (desktop alerts)
sound_manager.py - Audio feedback for operations
login_system.py - User authentication
auth_manager.py - Role-based access control
update_checker.py - Software update notifications
system_tray.py - System tray integration
translations.py - Multi-language support
legal_terms.py - Terms & conditions management
monitoring_page.py - Real-time operation monitoring
tamper_proof_page.py - Tamper detection UI
ai_assistant.py - AI-powered recommendations
algorithm_recommender.py - Algorithm suggestions

7. UI/UX SOLUTIONS & FIXES

7.1 File List Display Issue

Problem: File names not visible in QListWidget

Root Causes:

1. Missing color property in stylesheet
2. Item height too small for text display
3. No text wrapping enabled
4. Layout stretch factor causing overlap

Solutions Applied:

1. Added explicit color in stylesheet: color: #c9d1d9
2. Increased min-height to 48px (from 28px)
3. Enabled word wrap: setWordWrap(True)
4. Changed layout stretch to 0 (no expansion)
5. Set maximum height: setMaximumHeight(500)
6. Added visual separator between list and buttons

Resulting Styles:

- Dark background: #0d1117
- Light text: #c9d1d9
- Hover color: #00e676
- Selection: #147efb background
- Item padding: 16px 18px
- Line height: 1.5 for readability

7.2 Layout & Spacing Improvements

Changes Made:

1. Icon sizes standardized to 18x18px
2. Minimum button heights set to 36px
3. Spacing between elements: 8px
4. Dialog box minimum height: 120px
5. Main window minimum: 800x600px

Visual Hierarchy:

- Title labels: 14pt, bold, light color
- Section labels: 11pt, #888888
- Input fields: Dark background with border
- Buttons: Padded, with hover effects

8. THREADING & PERFORMANCE

8.1 Threading Model

Problem Being Solved:

Wiping large files can take significant time. Without threading, the UI would freeze, making the application unresponsive.

Solution: QThread with Signals/Slots

1. WipeWorker class runs in separate thread
2. Emits progress signals to main thread
3. Main thread updates UI via signal handlers
4. Non-blocking architecture

Implementation Pattern:

- WipeWorker(QObject) with custom signals
- progress_updated signal: emits (current, total, speed)
- finished signal: emits success/error status
- Main thread runs QThread: thread.start()
- UI remains responsive during wipe operations

8.2 Performance Characteristics

Chunk-Based Processing

- 8192 bytes (8KB) chunks read/written at once
- Reduces memory usage for large files
- Allows progress updates every ~8KB

Speed Estimates (Per Algorithm):

- Simple: ~50-100 MB/sec on modern SSD
- DoD: ~15-30 MB/sec (3x passes)
- NIST: ~50-100 MB/sec (SSD optimized)
- Gutmann: ~7-15 MB/sec (7x passes)
- Crypto: ~50-100 MB/sec (AES-NI hardware)

Optimization Techniques:

- Hardware AES acceleration (AES-NI)
- Buffer-based I/O (not byte-by-byte)
- Proper file system journal flushing
- Parallel processing for batch operations

9. SECURITY & COMPLIANCE

9.1 Encryption & Cryptography

Algorithms Used:

- AES-256: Industry standard encryption
- SHA-256: Hash function for verification
- `secrets.token_bytes()`: Cryptographically secure RNG
- PBKDF2: Key derivation for passwords

Key Management:

- Keys generated fresh for each operation
- No key persistence (keys destroyed after use)
- Hardware random number generator when available

Compliance Standards Supported:

- GDPR: Right to erasure compliance
- HIPAA: Protected health information destruction
- PCI-DSS: Payment card data elimination
- DoD 5220.22-M: Military data disposal standards
- NIST SP 800-88: Data sanitization guidelines

9.2 Audit & Logging

Tamper-Proof Audit Chain:

- Hash-chain implementation (blockchain pattern)
- Each log entry includes hash of previous entry
- Modification detection: any tampering breaks chain

Logged Information:

- Timestamp (precise to millisecond)
- User account (Windows integrated)
- Files/folders affected
- Algorithm used
- Success/failure status
- Duration and speed
- Device/media information

Audit Report Generation:

- PDF certificates via ReportLab
- Email delivery capability
- Compliance statement generation
- Digital signatures

10. BUILD & DEPLOYMENT GUIDE

10.1 Development Setup

Step 1: Clone/Download Project

- Ensure all project files are in one directory

Step 2: Create Virtual Environment (Optional)

- `python -m venv venv`
- `.\venv\Scripts\activate` (Windows)

Step 3: Install Dependencies

- `pip install -r requirements.txt`

Step 4: Run in Development

- `python secure_wipe_desktop.py`
- Opens GUI, ready for testing

Step 5: Build Executable

- `python build_exe.py`
- Generates `./dist/SecureWipe.exe`

10.2 Build Configuration

build_exe.py Details:

- Uses PyInstaller under the hood
- One-file executable (all dependencies bundled)
- Compresses Python bytecode
- Strips unneeded symbols
- Windows icon and version metadata included

Output Structure:

- `./dist/SecureWipe.exe` (main executable)
- `./build/` (temporary build artifacts)

11. FUTURE ENHANCEMENTS & ROADMAP

11.1 Planned Features

Version 3.0+ Roadmap:

Data Recovery Prevention

- Cluster Tip Wiper: Wipe NTFS cluster tips
- MFT Analysis: Master File Table sanitization
- Slack Space: Wipe partition slack space

Advanced Scheduling

- Recurring schedules (daily, weekly, monthly)
- Smart scheduling (during off-peak hours)
- Event-triggered wiping (USB insertion, etc.)

Cloud Integration

- OneDrive/SharePoint cloud file destruction
- Backup system integration
- Version history deletion

Management Features

- Central management console
- Multi-device deployment
- Remote monitoring dashboard

Platform Expansion

- Linux/Ubuntu support
- macOS support
- Web-based interface

11.2 Technical Debt & Optimization

Performance Improvements

- C++ backend for critical algorithms (via ctypes)
- SIMD vectorization for overwrite operations
- GPU acceleration for crypto operations

Code Quality

- Full unit test suite (pytest)
- Integration tests for all algorithms
- Security audit by third party
- Code coverage >80%

User Experience

- Dark/light theme toggle
- Customizable keyboard shortcuts
- Multi-language localization
- Accessibility improvements (WCAG 2.1)

CONCLUSION

The Secure Data Wiping System represents a comprehensive solution for government-grade data destruction. Built with modern Python technologies (PyQt6, cryptography), it provides:

- ✓ Military-grade wiping algorithms (DoD, NIST, Gutmann)
- ✓ Mathematically secure crypto-erase option (AES-256)
- ✓ Complete audit trail with tamper detection
- ✓ Compliance with GDPR, HIPAA, PCI-DSS standards
- ✓ Modern, responsive UI with threading
- ✓ Enterprise-ready with batch processing
- ✓ Extensive feature set (scheduling, verification, reporting)

All code follows security best practices, with proper error handling, logging, and validation throughout. The modular architecture enables future expansion and feature additions without compromising security or stability.

Document Generated: March 23, 2026 17:33:54

Secure Data Wiping System v2.0

© 2024 - Professional Data Destruction